

JAVA PROGRAMMING

Chapter 24: Lambdas & Functional Interfaces

Personal Notes

Lambda expressions are the single biggest change Java has had since generics. They were added in Java 8 and changed how Java code looks — turning multi-line classes into one-line expressions and making functional-style programming possible.

You've already used them in earlier chapters — every `Comparator.comparing(s -> s.marks)` was a lambda. This chapter unpacks what they really are, why they exist, and how to use them everywhere.

1. What is a Lambda Expression?

A lambda expression is a SHORT, ANONYMOUS FUNCTION that you can pass around like a value. It has no name, no class — just a list of parameters and a body.

The Basic Idea

A method:	A lambda:
<pre>public int add(int a, int b) { return a + b; }</pre>	<pre>(a, b) -> a + b</pre>
has a name, return type, modifiers	no name, no return type, just inputs → output

In one line, a lambda says: "given these inputs, here's what to do." That's it.

2. Why Lambdas? (Before vs After)

Before lambdas, every "small piece of behavior" you wanted to pass to a method had to be wrapped in a verbose anonymous class. Java 8 cleaned this up dramatically.

Before — Anonymous Class (Java 7 and earlier)

```
Collections.sort(students, new Comparator<Student>() {
    @Override
    public int compare(Student a, Student b) {
        return a.marks - b.marks;
    }
});
```

After — Lambda (Java 8+)

```
Collections.sort(students, (a, b) -> a.marks - b.marks);
```

Both do the EXACT same thing. The lambda is just dramatically less code — and it reads more like the actual intent: "sort by comparing marks."

Key insight: Lambdas don't add new power to Java. They make EXISTING patterns much cleaner. Anywhere you used to write an anonymous class with one method, you can now use a lambda.

3. Lambda Syntax

A lambda expression has three parts: parameters, an arrow `->`, and a body.

```
(parameters)  ->  body
      |           |
      v           v
    input list  what to do with them
```

3.1 The Variations

No parameters

```
() -> System.out.println("Hello")
```

One parameter (parentheses optional)

```
x -> x * 2
(x) -> x * 2      // same thing – parentheses optional for single
param
```

Multiple parameters

```
(a, b) -> a + b
(int a, int b) -> a + b    // explicit types (optional)
```

Multi-line body (use braces and return)

```
(a, b) -> {
    int sum = a + b;
    System.out.println("Adding " + a + " + " + b);
    return sum;
}
```

Single expression body (no braces, no return)

```
(a, b) -> a + b      // implicit return – the expression IS the return
```

The rule: Single expression? No braces, no return — Java just returns the expression. Multiple statements? Wrap in braces and use return like a normal method. Use single-expression style whenever you can — it's the whole point of lambdas.

4. Functional Interfaces

Here is the key fact that makes lambdas work in Java: a lambda is always assigned to a FUNCTIONAL INTERFACE. A functional interface is simply an interface with EXACTLY ONE abstract method.

```
// This is a functional interface — exactly one abstract method
interface Greeting {
    void sayHello();
}

// We can assign a lambda to it
Greeting g = () -> System.out.println("Hello!");
g.sayHello();    // prints: Hello!
```

The lambda's parameter list and body must match the interface's single abstract method. Java figures out from the interface what the method's signature should be.

4.1 The @FunctionalInterface Annotation

You can mark an interface as functional explicitly. If you do, the compiler will check that it has exactly one abstract method — catching mistakes early.

```
@FunctionalInterface
interface Calculator {
    int compute(int a, int b);
}

// Usage with different lambdas:
Calculator add = (a, b) -> a + b;
Calculator sub = (a, b) -> a - b;
Calculator mul = (a, b) -> a * b;

System.out.println(add.compute(5, 3));    // 8
System.out.println(sub.compute(5, 3));    // 2
System.out.println(mul.compute(5, 3));    // 15
```

Important: The @FunctionalInterface annotation is optional but recommended. It documents your intent AND lets the compiler catch the moment someone accidentally adds a second abstract method, breaking lambda compatibility.

4.2 Default and Static Methods Are Allowed

A functional interface can have any number of default methods or static methods — only the ABSTRACT method count must be exactly one.

```
@FunctionalInterface
interface Greeter {
    void greet(String name);           // single abstract – required

    default void greetLoudly(String name) {    // default – OK
        greet(name.toUpperCase());
    }

    static Greeter friendly() {           // static – OK
        return n -> System.out.println("Hi " + n + "!");
    }
}
```

5. Built-in Functional Interfaces

Java 8 ships with a package called `java.util.function` that contains ready-made functional interfaces for the most common patterns. You'll use these constantly — so it's worth knowing the main ones.

Interface	Signature	Use Case
Predicate<T>	$T \rightarrow \text{boolean}$	Test a value: $x \rightarrow x > 0$
Function<T,R>	$T \rightarrow R$	Transform a value: $x \rightarrow x * 2$
Consumer<T>	$T \rightarrow \text{void}$	Use a value with no return: $x \rightarrow \text{println}(x)$
Supplier<T>	$() \rightarrow T$	Produce a value: $() \rightarrow \text{Math.random}()$
BiFunction<T,U,R>	$(T,U) \rightarrow R$	Two inputs, one output: $(a,b) \rightarrow a+b$
BiPredicate<T,U>	$(T,U) \rightarrow \text{boolean}$	Two inputs, boolean: $(a,b) \rightarrow a.\text{equals}(b)$
UnaryOperator<T>	$T \rightarrow T$	Same type in/out: $x \rightarrow x*x$
BinaryOperator<T>	$(T,T) \rightarrow T$	Two inputs and output, same type

5.1 Predicate — Yes/No Tests

```
import java.util.function.Predicate;
```

```

Predicate<Integer> isPositive = x -> x > 0;
Predicate<String> isEmpty    = s -> s.isEmpty();

isPositive.test(5);          // true
isPositive.test(-3);         // false
isEmpty.test("");           // true

// Useful combinators:
Predicate<Integer> isEven    = x -> x % 2 == 0;
Predicate<Integer> isBig     = x -> x > 100;

isEven.and(isBig).test(150); // true (even AND big)
isEven.or(isBig).test(7);    // false (neither)
isEven.negate().test(7);     // true (not even)

```

5.2 Function — Transform Values

```

import java.util.function.Function;

Function<Integer, Integer> square = x -> x * x;
Function<String, Integer> length = s -> s.length();

square.apply(5);          // 25
length.apply("Hello");    // 5

// Chaining
Function<Integer, Integer> doubled = x -> x * 2;
Function<Integer, Integer> chained = square.andThen(doubled);
chained.apply(3);         // 18 (3 squared = 9, then doubled = 18)

```

5.3 Consumer — Do Something with a Value

```

import java.util.function.Consumer;

Consumer<String> print = s -> System.out.println(s);
Consumer<Student> showStudent = s -> System.out.println(s.name + " - " + s.marks);

print.accept("Hello");

// Used a lot with forEach:
List<String> names = Arrays.asList("Aman", "Riya", "Karan");
names.forEach(print);

```

5.4 Supplier — Produce Values

```

import java.util.function.Supplier;

Supplier<String> greeting = () -> "Hello, World!";

```

```
Supplier<Double> randomValue = () -> Math.random();

System.out.println(greeting.get());    // Hello, World!
System.out.println(randomValue.get()); // 0.7541... (varies)
```

5.5 BiFunction — Two Inputs

```
import java.util.function.BiFunction;

BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;
BiFunction<String, String, String> concat = (a, b) -> a + " " + b;

add.apply(3, 4);           // 7
concat.apply("Hello", "Java"); // Hello Java
```

6. Method References — The :: Operator

When a lambda just calls an existing method without modifying anything, you can replace it with a METHOD REFERENCE using the :: operator. Even shorter, even cleaner.

Before — Lambda

```
names.forEach(name -> System.out.println(name));
```

After — Method Reference

```
names.forEach(System.out::println);
```

Both call `System.out.println` on each name. The :: version just doesn't repeat the parameter.

6.1 The Four Kinds of Method References

Type	Syntax	Example
Static method	<code>ClassName::staticMethod</code>	<code>Integer::parseInt</code>
Instance method (specific object)	<code>obj::method</code>	<code>System.out::println</code>
Instance method (any object of type)	<code>Class::method</code>	<code>String::toUpperCase</code>
Constructor	<code>ClassName::new</code>	<code>ArrayList::new</code>

Practical Examples

```
// Lambda → Method Reference

// Static method
```

```

List<String> nums = Arrays.asList("1", "2", "3");
nums.stream().map(s -> Integer.parseInt(s));    // lambda
nums.stream().map(Integer::parseInt);           // method ref

// Instance method on a specific object
PrintStream out = System.out;
names.forEach(n -> out.println(n));             // lambda
names.forEach(out::println);                   // method ref

// Instance method on any object of a type
List<String> words = Arrays.asList("hi", "bye");
words.stream().map(s -> s.toUpperCase());       // lambda
words.stream().map(String::toUpperCase);       // method ref

// Constructor reference
Supplier<ArrayList<Integer>> factory = () -> new ArrayList<>(); //
lambda
Supplier<ArrayList<Integer>> factory2 = ArrayList::new;         //
method ref

```

7. Lambdas in Action

Now let's see where lambdas really shine — in everyday Java code.

7.1 Sorting

```

List<Student> students = ...;

// Sort by marks
students.sort((a, b) -> a.marks - b.marks);

// Cleaner version using Comparator.comparing
students.sort(Comparator.comparing(s -> s.marks));

// Even cleaner with method reference
students.sort(Comparator.comparing(Student::getMarks));

```

7.2 forEach

```

List<String> names = Arrays.asList("Aman", "Riya", "Karan");

names.forEach(name -> System.out.println(name));
names.forEach(System.out::println);    // even shorter

```

7.3 removeIf

```

List<Integer> nums = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5, 6));
nums.removeIf(x -> x % 2 == 0);    // remove all even numbers

```

```
System.out.println(nums); // [1, 3, 5]
```

7.4 Custom Functional Interfaces

```
@FunctionalInterface
interface MathOp {
    int apply(int a, int b);
}

public class Main {
    static int compute(int a, int b, MathOp op) {
        return op.apply(a, b);
    }

    public static void main(String[] args) {
        System.out.println(compute(5, 3, (a, b) -> a + b)); // 8
        System.out.println(compute(5, 3, (a, b) -> a * b)); // 15
        System.out.println(compute(5, 3, Math::max)); // 5
    }
}
```

8. Variable Capture (Closures)

Lambdas can use variables from the surrounding code — this is called CAPTURE. But there's a strict rule: captured local variables must be EFFECTIVELY FINAL (never reassigned after being set).

```
int multiplier = 5;
Function<Integer, Integer> times = x -> x * multiplier; // captures 'multiplier'
System.out.println(times.apply(10)); // 50

// multiplier = 6; // ERROR if you uncomment – would break the rule
```

Why this rule exists: Lambdas can outlive the method they're created in. If the local variable could change later, the lambda would get unpredictable values. "Effectively final" means "never reassigned" — Java enforces it to keep things safe.

What CAN be captured freely

- Instance fields (this.x can change freely)
- Static fields
- Method parameters (they're effectively final once the method starts)
- Local variables that are never reassigned after their initial value

9. Lambda vs Anonymous Class — Comparison

Aspect	Lambda	Anonymous Class
Syntax	Short, expression-like	Verbose, full class body
this keyword	Refers to enclosing class	Refers to anonymous class itself
Multiple methods	Cannot — only one abstract method	Can override multiple methods
State	Stateless usually	Can have fields
Best for	Single-method interfaces (functional)	Multi-method or complex behavior
Performance	Slightly faster (invokedynamic)	Slightly slower (extra class file)

10. Common Mistakes

Mistake 1: Trying to use lambdas with non-functional interfaces

```
interface MultiMethod {  
    void doA();  
    void doB();  
}  
  
MultiMethod m = () -> System.out.println("hi");    // ERROR — 2 abstract  
methods
```

Lambdas work ONLY with interfaces that have exactly ONE abstract method. If the interface has more, you'll need a regular class or anonymous class.

Mistake 2: Reassigning captured variables

```
int count = 0;  
Runnable r = () -> System.out.println(count);  
count = 5;    // ERROR — 'count' is not effectively final
```

Mistake 3: Using == inside lambdas with Strings

```
names.removeIf(s -> s == "test");    // WRONG — reference compare  
names.removeIf(s -> s.equals("test")); // correct
```

Mistake 4: Forgetting return in multi-line lambda

```
(a, b) -> {  
    int sum = a + b;  
    sum;    // ERROR — missing return!  
}
```

```
(a, b) -> {
    int sum = a + b;
    return sum;    // correct
}
```

If you use braces, you also need an explicit return. Only single-expression lambdas auto-return.

Mistake 5: Wrong number of parameters

```
// Comparator needs (T, T) -> int
Comparator<Integer> c = a -> a;    // ERROR - needs two parameters
```

Mistake 6: Overusing lambdas for complex logic

Lambdas should be short and clear. If you find yourself writing a 30-line lambda with deeply nested logic, extract it into a real method. Long lambdas hurt readability more than they help.

11. Best Practices

- Keep lambdas short — one line if possible, three lines at most
- Use method references (::) when the lambda just calls one method
- Prefer built-in functional interfaces (Predicate, Function, Consumer) over custom ones
- Add @FunctionalInterface on custom functional interfaces — catches mistakes early
- Make sure captured variables are effectively final — don't reassign them
- Use lambdas with collections — forEach, removeIf, sort, stream operations
- Don't sacrifice clarity for shortness — a clear lambda beats a clever one

12. Quick Reference

Syntax / Concept	Meaning
() -> body	Lambda with no parameters
x -> body	Lambda with one parameter (parens optional)
(a, b) -> body	Lambda with multiple parameters
(a, b) -> { ... }	Multi-line body, requires explicit return
@FunctionalInterface	Marks an interface as having exactly one abstract method
Predicate<T>	T -> boolean (yes/no tests)
Function<T,R>	T -> R (transformations)
Consumer<T>	T -> void (side effects, like printing)
Supplier<T>	() -> T (produces values)

Syntax / Concept	Meaning
BiFunction<T,U,R>	(T,U) -> R (two-input transformation)
Class::method	Method reference
Class::new	Constructor reference
Effectively final	Captured variable rule — never reassign

13. Practice Problems

Try each one without copying code from earlier sections. The goal is to internalize the syntax.

1. Create a functional interface `Calculator` with `int operate(int a, int b)`. Use lambdas to add, subtract, multiply, and divide two numbers.
2. Use a `Predicate<Integer>` to test if a number is even. Then negate it to test odd.
3. Use `Function<String, Integer>` to convert a `String` to its length.
4. Use a `Consumer<String>` to print each name in a list with `forEach()`.
5. Use a `Supplier<Double>` that returns `Math.random()` each time.
6. Use `Comparator.comparing(...)` with a method reference to sort a list of `Students` by name.
7. Use `removeIf()` with a lambda to remove all negative numbers from a list.
8. Use `BiFunction<Integer, Integer, Integer>` to find max, min, and average of two numbers.
9. Chain two `Predicates` with `.and()` — "positive AND even."
10. Convert this anonymous class to a lambda: `new Runnable() { public void run() { System.out.println("go"); } }`
11. Use `Function.andThen()` to compose two transformations: square then add 10.
12. Create a functional interface `TriFunction<A,B,C,R>` for 3-arg functions. Use it to add three numbers.

14. Final Takeaway

Lambdas are not just a syntactic shortcut — they unlocked a whole functional style in Java. Combined with the Streams API (next chapter), they let you express data transformations in ways that used to require dozens of lines of loops and helpers.

If you can read and write lambdas fluently, modern Java code (Spring, Android, Streams, reactive frameworks) suddenly becomes much more approachable. Don't try to memorize every functional interface — just internalize the four core ones (`Predicate`, `Function`, `Consumer`, `Supplier`) and the rest will follow naturally from patterns.

Key Takeaway: A lambda is just a small function you can pass around as a value. It targets a functional interface (an interface with one abstract method). Master Predicate (boolean), Function (transform), Consumer (use it), and Supplier (produce one) — those four cover most real-world cases. Then use method references (::) wherever your lambda is just calling an existing method.

15. What's Next?

Lambdas are the building block of the next major topic — Streams. From here:

- Java Streams API — filter, map, reduce, collect on collections
- `Optional<T>` — handling "maybe null" the modern way
- Collectors — turning streams back into lists, maps, sets
- Parallel streams — leveraging multiple CPU cores
- File I/O with `Files.lines()` and Streams
- Java 9+ features — `var` keyword, modules, switch expressions